

The Info Pharma Ordering System (IPOS)



Centre for Software Reliability,
School of Informatics
City University,

London, 2026

Summary

This is a Requirements Specification and High-Level Design Document, a proposal to the managing director of Info Pharma Ltd, by team 4, Readers of Systems Dependability at the Centre for Software Reliability, City University.

Authors

Herman Sildnes
Joanna Ayeni
Leon Seferi
Kanzah Hussain
Vladyslav Semanyshym
Yamin Ahmed
Jenis Khorshed

Distribution

The Managing Director of Info Pharma Ltd, the development team at the Centre for Software Reliability at the School of Informatics, City University London

Contents

1 Preface.....	3
1.1 Purpose and Scope of the document.....	3
1.2 Intended audience and Reading Suggestions.....	3
1.3 History of the document.....	3
2 Introduction.....	4
3 Glossary.....	5
4 User Requirements.....	6
5 System requirements.....	7
5.1 Purpose and scope of the system.....	7
5.2 System models.....	7
5.2.1 User stories.....	7
5.2.1.1 IPOS-SA-ACC.....	7
5.2.1.2 IPOS-SA-CAT.....	8
5.2.1.3 IPOS-SA-ORD.....	8
5.2.1.4 IPOS-SA-RPT.....	8
5.2.2 Prioritised List of User Stories.....	8
6 Anticipated Evolution of System Requirements.....	9
6.1 Validation.....	9
6.2 Loyalty Cards - subject to change.....	9
7 Info Pharma High-Level Design.....	10
7.1 IPOS-SA.....	10
7.2 IPOS-CA.....	10
7.3 IPOS-PU.....	10
7.4 Software Architecture.....	10
7.4.1 Design diagram of subsystem IPOS-SA.....	11
7.5 Source Code (Python) for the interfaces of IPOS-SA.....	11
7.6 Testing the communications between subsystems.....	12
8 Appendicies.....	13
8.1 Initial Statement of requirements by the client.....	13
8.2 Hardware (minimal and optimal configuration).....	13
8.2.1 minimal hardware configuration.....	13
8.2.2 optimal hardware configuration.....	13
9 Index.....	14

1 Preface

1.1 Purpose and Scope of the document

This is a Requirements Specification for one of the subsystems of IPOS, IPOS-SA, and the High-Level Design document for the IPOS software, which includes IPOS-SA and two other subsystems, IPOS-CA and IPOS-PU. The document covers the functional requirement of IPOS-SA and the high-level design of the entire IPOS system showing the IPOS architecture composed of three subsystems and the mechanisms of interaction between them. The document is provided in response to the initial statement of requirements by Info Pharma Ltd, the client.

The document explains what the subsystem shall do, and how its subsystems might interact with one another, but does not cover details about how the functionality of IPOS-SA subsystem will be implemented. Various UML models are used together with fragments of software code to convey in more detail to system developers the expected behaviour of the IPOS-SA subsystems and especially how this subsystem will interact with the other subsystems of IPOS.

1.2 Intended audience and Reading Suggestions

The Managing Director of Info Pharma Ltd, the system analysts and architects at the Centre of Software Reliability (CSR), City University London

Some of the sections are intended for the CSR team only, which is clearly stated in the document.

1.3 History of the document

This is v1.0 of the document.

2 Introduction

IPOS will allow Info Pharma Ltd to automate their business process in three interrelated areas: i) merchant account management, ii) ordering products and iii) selling inventory to the general public. By introducing an online ordering system the merchants will be able to make pharmaceutical orders and subsequently sell these products in their shops.

The three areas of planned automation are expected to be covered by three subsystems: IPOS-SA, IPOS-CA and IPOS-PU, respectively. This document is focused on IPOS-SA. The other subsystems are covered in separate documents.

In addition to the services offered to merchants, IPOS-SA allows Info Pharma to maintain an inventory of products which merchants can then purchase. Thus allowing merchants to directly buy products and configure discount plans for their purchases and set up inventory for their own business.

IPOS-SA will also send invoices and reports of products bought and profit generated when requested by the merchant.

IPOS-SA will interact with the other subsystems, IPOS-CA and IPOS-PU, too.

3 Glossary

Actor	A human or an external system interacting with IPOS e.g. merchant, administrator, manager, etc
CSR	Centre for software reliability
IPOS	Info Pharma Ordering System
UML	Unified Modelling language
User Story	A basic interaction of a system with an actor

4 User Requirements

This section is left intentionally blank

5 System requirements

5.1 Purpose and scope of the system

This section provides a more detailed specification of the required functionality (known as system requirements) for IPOS-SA.

The intended audience are the *system architectures, designers and software developers* responsible for the implementation of IPOS. Since we use UML, which requires a degree of specific expert knowledge, we do not expect this section to be immediately useful to the Info Pharma staff but would like to encourage them to review and comment, if possible.

5.2 System models

5.2.1 User stories

The user stories for the IPOS-SA subsystem are shown below in table format.

5.2.1.1 IPOS-SA-ACC

This package provides functionality needed for setting up accounts for the users of IPOS-SA

Title: merchant filling in details for account	Priority: essential	User Story ID: SA-ACC-01
User Story: As a merchant, I want to fill in details, So that I can create an account on the system		
Acceptance Criteria: Given the system prompts the merchant to input their account details, When the merchant fills in their details, Then the system should activate an account based on these details the merchant can log into.		

Title: merchant implementing discount plans	Priority: important	User Story ID: SA-ACC-02
User Story: As a merchant, I want to implement two types of discount plans, So that my account will be valid to place orders.		
Acceptance Criteria: Given the system prompts the merchant to input their discount plans,		

Commercial-in-Confidence

When they have input at least two plans (one fixed and one flexible),
Then as long as all other details have been added correctly, the subsystem should activate this account as valid and allow orders to be taken.

Title: manager altering
account details of accounts

Priority: important

User Story ID: SA-ACC-03

User Story:

As a manager,
I want have access to merchant accounts,
So that I can alter credit limits and discount plans.

Acceptance Criteria:

Given the manager has logged on,
When they select a merchant account they wish to alter,
Then they have access to changing details corresponding with this account.

Title: manager changing
state of merchant account

Priority: important

User Story ID: SA-ACC-04

User Story:

As a manager,
I want to have access to merchant accounts,
So that I can change the state of the account.

Acceptance Criteria:

Given the manager has logged on and they have selected the account whose state they wish to change,
When the merchant has made a payment within the 15-30 day limit (if the account has been suspended) and over 30 day limit (if the account has been in default),
Then the manager has the ability to change the state of the account from in default or suspended back to normal.

Title: administrator making a
merchant account

Priority: essential

User Story ID: SA-ACC-05

User Story:

As an administrator,
I want to create a merchant account,
So that merchants can place orders and access relevant IPOS-SA functionality.

Acceptance Criteria:

Given all required details are provided by the merchant
When the administrator submits these details to the system,
Then the merchant account is created.

Commercial-in-Confidence

Title: administrator accessing IPOS-SA	Priority: essential	User Story ID: SA-ACC-06
User Story: As an administrator, I want to have access to all functionality of IPOS-SA, So that I can access user accounts of the merchants, including catalogs that they can view.		
Acceptance Criteria: Given the user is an admin, When they log into the system, Then they should have access to all IPOS-SA functionality.		

Title: invalid merchant details given to administrator	Priority: important	User Story ID: SA-ACC-07
User Story: As an administrator, I want to prevent invalid merchant accounts, So that only valid and usable accounts are able to log into the system.		
Acceptance Criteria: Given some of the required information is missing from the merchant application, When the administrator attempts to create the merchant account, Then the account is not created and a reason should be stated.		

5.2.1.2 IPOS-SA-CAT

This package provides the functionality necessary to maintain a catalog of goods available to the merchants to order from.

Title: add and maintain catalog entries	Priority: essential	User Story ID: SA-CAT-01
User Story: As an Operations Manager, I want to add and maintain catalog entries with full product specifications, So that the product information is accurately recorded and available for merchants to view and order from.		
Acceptance Criteria: Given the manager has logged in, When they add a valid product to the catalog, Then merchants should be able to see full product specifications.		

Title: stock limit for products	Priority: important	User Story ID: SA-CAT-02
--	----------------------------	---------------------------------

Commercial-in-Confidence

User Story:

As an Operations Manager,

I want configure a specific “stock limit” (minimum threshold) for every catalog item,

So that the system can automatically trigger low-stock warnings when inventory levels fall below the defined threshold.

Acceptance Criteria:

Given the stock limit of a product has been set,

When a merchant tries to purchase an amount of this product below this limit,

Then the system should say this is an invalid order and not accept it.

Title: update availability count

Priority: important

User Story ID: SA-CAT-03

User Story:

As a warehouse staff,

I want to increment the ‘availability’ count for specific items when new shipments are delivered,

So that the stock levels in the system accurately reflect the most current inventory available for sale.

Acceptance Criteria:

Given this product exists in the catalogue,

When an order of this product occurs,

Then the system should automatically update to reflect this change in stock.

Title: automated low stock warning

Priority: good-to-have

User Story ID: SA-CAT-04

User Story:

As an Operations Manager/ administrators,

I want to receive an automated warning on my home screen for any item where availability falls below the stock limit,

So that a timely action to replenish stock and avoid running out of products can happen.

Acceptance Criteria:

Given the manager/admin are logged onto the system,

When the stock of a product is low,

Then the system should notify the user of its low stock as a warning message.

Title: generating a low stock report

Priority: good-to-have

User Story ID: SA-CAT-05

User Story:

As an Operations Manager,

I want to generate a “low stock level report” that calculates a “recommended min order”,

So that i can efficiently manage inventory replenishment and ensure optimal stock levels are

Commercial-in-Confidence

maintained.

Acceptance Criteria:

Given a manager is logged onto the system,

When they generate a low stock report,

Then the system should calculate a recommended min order so stock levels are maintained.

Title: real-time availability

Priority: good-to-have

User Story ID: SA-CAT-06

User Story:

As a Merchant,

I want to see the real-time availability of items while filling out my electronic order form,

So that i know which products are in stock before placing an order and can make informed purchasing decisions.

Acceptance Criteria:

Given a merchant is logged onto the system,

When they look at a product within the catalogue,

Then the real-time availability should be reflected of this object.

5.2.1.3 IPOS-SA-ORD

This package provides the functionality necessary for a merchant to be able to place a new order (choosing items from the current catalog) and track the status of their orders, outstanding unpaid balances, etc.

Title: placing an order

Priority: essential

User Story ID: SA-ORD-01

User Story:

As an Merchant,

I want to be able to place orders on the system,

So that a new order is created on the system.

Acceptance Criteria:

Given the merchant has products in their basket,

When they place an order,

Then the system should generate a new order of these products and update the order database.

Title: reducing stock

Priority: essential

User Story ID: SA-ORD-02

User Story:

As the system,

I want to reduce stock of a product when an order is made,

So that the current stock reflects the inventory stock.

Commercial-in-Confidence

Acceptance Criteria:

Given an order has been placed on this product,

When the order is accepted,

Then the system should reduce the stock by the amount of product purchased.

Title: tracking orders	Priority: important	User Story ID: SA-ORD-03
User Story: As an Merchant, I want to be able to track orders I have placed in real-time, So that I can see dispatch and delivery details.		
Acceptance Criteria: Given the merchant has placed an order, When they look at their current orders, Then the system should display details about their current orders.		

Title: raising invoices	Priority: good-to-have	User Story ID: SA-ORD-04
User Story: As an Merchant, I want to be able to raise an invoice, So that I can see data to do with financial details of an order I have made.		
Acceptance Criteria: Given the merchant has placed an order, When they raise an invoice, Then the system should generate an invoice of this order placed.		

5.2.1.4 IPOS-SA-RPT

This package provides the functionality for generating various reports via searching the system database.

Title: generating a global turnover report	Priority: good-to-have	User Story ID: SA-RPT-01
User Story: As a Director of operations, I want to be able to generate a high-level turnover report for the entire company, So that I can analyse total sales performance across all merchants and product categories.		
Acceptance Criteria: Given director of operations has logged on, When they request to generate the global turnover report, Then the system should generate a report in the high-level summary dashboard providing		

Commercial-in-Confidence

functionality for it to be downloaded as a spreadsheet.

Title: generating a period turnover report	Priority: good-to-have	User Story ID: SA-RPT-02
User Story: As a Director of operations, I want to be able to generate a high-level turnover report for the entire company for a specific period of time, So that I can analyse total sales performance across all merchants and product categories of this period.		
Acceptance Criteria: Given director of operations has logged on, When they request to generate the period turnover report and have input the period they want to see, Then the system should generate a report in the high-level summary dashboard providing functionality for it to be downloaded as a spreadsheet.		

Title: generating a list of orders for a specific merchant	Priority: good-to-have	User Story ID: SA-RPT-03
User Story: As a Director of operations, I want to be able to generate a report of orders for an individual merchant, So that I can analyse sales performance from this merchant.		
Acceptance Criteria: Given director of operations has logged on, When they request to generate the order report and have specified a merchant, Then the system should generate a report in the high-level summary dashboard providing functionality for it to be downloaded as a spreadsheet based on this merchant.		

Title: generating a list of orders for a specific merchant for a specific time period	Priority: good-to-have	User Story ID: SA-RPT-04
User Story: As a Director of operations, I want to be able to generate a report of orders for an individual merchant in a specific time period, So that I can analyse sales performance from this merchant during this time period.		
Acceptance Criteria: Given director of operations has logged on, When they request to generate the order report and have specified a merchant and time period,		

Commercial-in-Confidence

Then the system should generate a report in the high-level summary dashboard providing functionality for it to be downloaded as a spreadsheet based on this merchant within this period.

Title: generating a report on specific merchant activity

Priority: good-to-have

User Story ID: SA-RPT-05

User Story:

As a Director of operations,

I want to be able to generate a report on the activity of a merchant for a given period of time,

So that I can analyse the activity of this merchant.

Acceptance Criteria:

Given director of operations has logged on,

When they request to generate the activity report and have specified a merchant and time period,

Then the system should generate a report in the high-level summary dashboard providing functionality for it to be downloaded as a spreadsheet based on this merchant within this period.

Title: generating a report on stock sold

Priority: good-to-have

User Story ID: SA-RPT-06

User Story:

As a Director of operations,

I want to be able to generate a report on stock sold during a period of time,

So that I can analyse product performance during this time period.

Acceptance Criteria:

Given director of operations has logged on,

When they request to generate the stock report and specify a period of time,

Then the system should generate a report in the high-level summary dashboard providing functionality for it to be downloaded as a spreadsheet based on the stock turnover this period.

5.2.2 Prioritised List of User Stories

The list below prioritises the user stories included in the system requirements according to their importance in IPOS-SA. CSR management will use the assigned priorities to decide on the order of implementing the functionality specified in the user stories according to the resources available: the **essential** user stories will be implemented first, then the user stories ranked as **important** will follow. The user stories ranked as **good-to-have** will be implemented last provided, the resources (manpower and time) are available to complete their implementation.

Commercial-in-Confidence

Rank	User Story ID	User Story name	US priority	Justification
1	SA-ACC-06	Admin accessing IPOS-SA	essential	Admin needs to be able to access IPOS-SA or other accounts cannot be created hence this is crucial to functionality
2	SA-ACC-01	Merchant filling in details for account	essential	Merchants need to be able to fill in details for an account to access IPOS-SA or no orders can be made on the system
3	SA-ACC-05	Admin making a merchant	essential	Admins need to then be able to create an account based on this information or merchants cannot use IPOS-SA functionality
4	SA-ACC-07	Invalid merchant details given to admin	important	Invalid accounts cannot be made on the system so this functionality needs to be added early on
5	SA-ACC-02	Merchant implementing discount plans	important	If a merchant cannot implement discount plan then their account will be invalid and hence they cannot make orders
6	SA-CAT-01	Adding and maintaining catalog entries	essential	If there are no products in a catalog then no orders can be made
7	SA-ORD-01	Placing an order	essential	This is core functionality of this system, however it is not the most important rank as account

Commercial-in-Confidence

				functionality is needed before customers can make orders
8	SA-ORD-02	Reducing stock	essential	If this is not implemented then stock will not reflect physical stock in warehouses and incorrect orders will be made based on stock Info Pharma does not have
9	SA-ACC-03	Manager altering account details for merchants	important	If a merchant needs to alter details, without this functionality that cannot happen
10	SA-ACC-04	Manager changing state of merchant account	important	If this functionality is not added, merchants will remain suspended or in default
11	SA-CAT-02	Stock limit for products	important	If this functionality is not added invalid orders will be made where orders exceed legal limits or stock availability
12	SA-CAT-03	Update availability count (for warehouse staff)	important	If this functionality is not added, warehouse staff will not know stock amount to then send a request and order more of the product
13	SA-CAT-06	Real-time availability (for merchants)	good-to-have	If this functionality is not added, orders may not be valid as products do not currently exist in warehouse

Commercial-in-Confidence

14	SA-ORD-03	Tracking orders	important	If this functionality is not added merchants will not be able to track their orders
15	SA-ORD-04	Raising invoices	good-to-have	If this functionality is not added there will be no record of purchase created for merchant or Info Pharma
16	SA-RPT-01	Generating a global turnover report	good-to-have	Required functionality from SA-RPT
17	SA-RPT-02	Generating a period turnover report	good-to-have	Required functionality from SA-RPT
18	SA-RPT-03	Generating a list of orders for a specific merchant	good-to-have	Required functionality from SA-RPT
19	SA-RPT-04	Generating a list of orders for a specific merchant in a specific time period	good-to-have	Required functionality from SA-RPT
20	SA-RPT-05	Generating a report on activity of a specific merchant	good-to-have	Required functionality from SA-RPT
21	SA-RPT-06	Generating a report on stock sold	good-to-have	Required functionality from SA-RPT
22	SA-CAT-04	Automated low stock warnings	good-to-have	Not as important but good to show managers what products need to be ordered
23	SA-CAT-05	Generating a low stock report	good-to-have	Not as important but good to show managers what products need to be ordered

6 Anticipated Evolution of System Requirements

We expect that IPOS-SA will be developed and deployed quickly. The operational experience from merchants and staff will inform the future decisions about the necessary changes. The envisaged likely changes to various parts of IPOS-SA are summarised below.

6.1 Validation

Currently the validation of account information provided during registration of their account will be checked by administrators in Info Pharma. In the future, however, we envisage that the validation may be done using 3rd party online services, which will require changes.

6.2 Enterprise SSO Login

In the future, a feature we envisage for the system is having an enterprise SSO log in where employees of a merchant can use their SSO authentication to log into the system and access company specific data and products.

7 Info Pharma High-Level Design

This section defines the high-level design on the entire IPOS software, which includes its architecture - the subsystems which it is made up of, and the communication interfaces between the subsystems. The subsystems envisaged are:

7.1 IPOS-SA

This subsystem will allow merchants to register with Info Pharma to view the catalog of products offered and make orders and discount plans.

7.2 IPOS-CA

This subsystem will allow merchants to access IPOS-SA-ORD, and contains a set of additional features to automate the operation of the merchant in dealing with their own customers via a desktop application.

7.3 IPOS-PU

This subsystem will allow merchants to sell goods to members of the public online via an online portal.

7.4 Software Architecture

The architecture of IPOS as a UML component diagram, developed with Visual Paradigm, Professional Edition, is shown on figure 1.

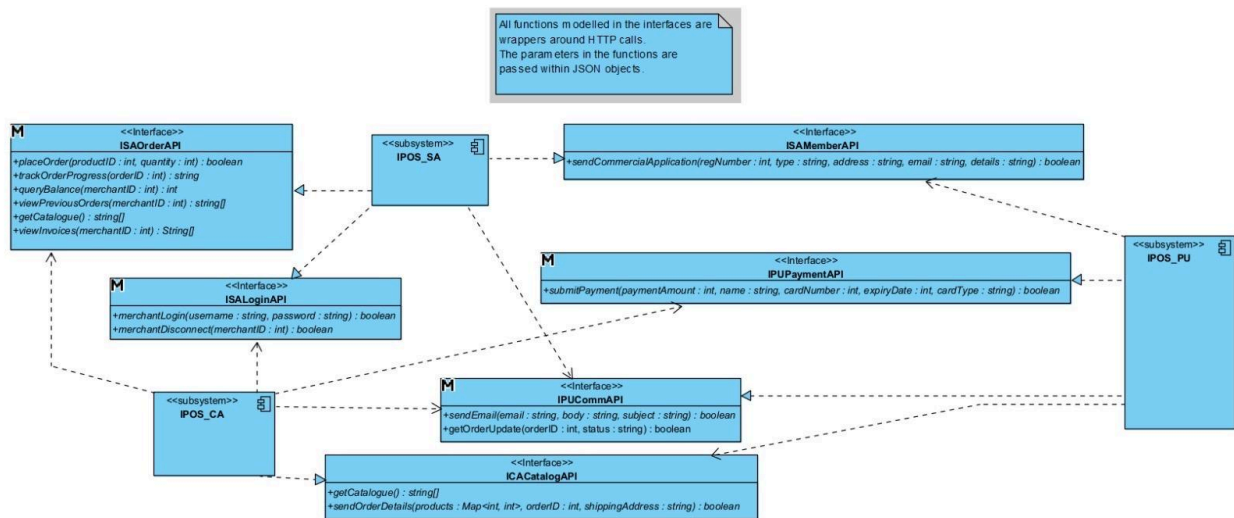


Figure 1. IPOS architecture as a UML component diagram. Each subsystem is modelled as a component with stereotype <<subsystem>>. The communication between the subsystems is

achieved via a set of assembly connectors (i.e. a pair of “provided” and “required” interfaces referring to the API offered by the respective subsystems) are provided by IPOS-SA, IPOS-CA and IPOS-PU, respectively and the APIs are used to access the functionality offered by one of the subsystems by the other two subsystems.

The methods defined in interfaces are explicitly shown with their full signatures. These are plausible public methods via which functionality of respective subsystems can be accessed. Note that changing the presentation of interfaces, changed the relationships of the interfaces too: the subsystems responsible to implement the interface has a “realisation” relationship with it, which implies that the respective subsystem will contain an implementation class realising the interface. The subsystems using the the API (i.e. for which the API is a “required” interface) now would have a dependency relationship with the interface (<<usage>> icon in Visual Paradigm should be used in the construction of the component diagram).

The IPOS architecture includes three subsystems and shows the interfaces necessary for these subsystems to interact with each other

7.4.1 Design diagram of subsystem IPOS-SA

Next, we elaborate the high-level design of IPOS-SA and provide details on the interfaces that this subsystem needs, and on the implementation of the subsystem’s required interface.

The class diagram of IPOS-SA subsystem is shown in figure 2.

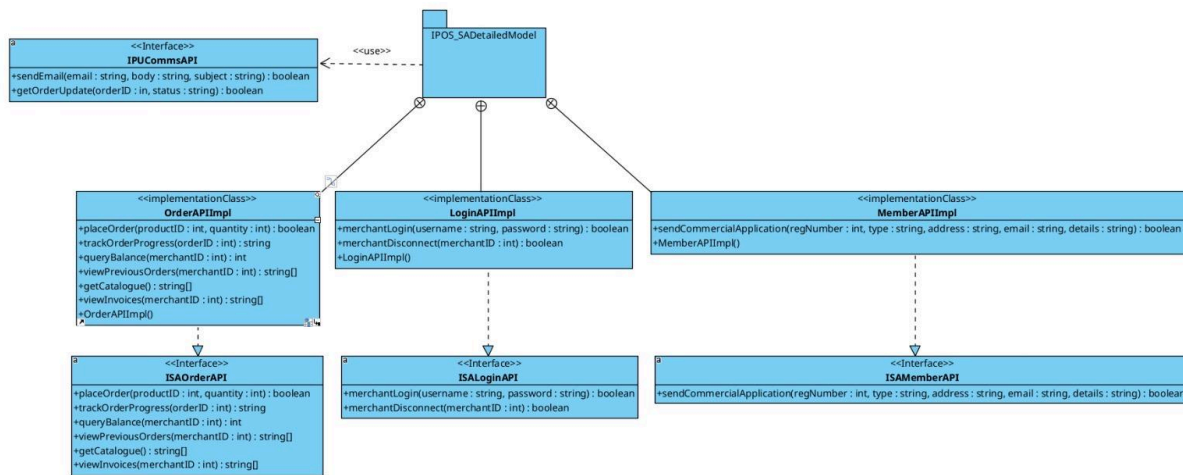


Figure 2. High-Level design of IPOS-SA

7.5 Source Code (Python) for the interfaces of IPOS-SA

Now that we have defined all interfaces and the implementation class, source code could be derived in Python for the interfaces and the implementation classes shown in Figure 2. The complete Python code of the interfaces and a skeleton of the implementation class are shown below:

```
public interface IPUCommsAPI {  
  
    /**  
     *  
     * @param email  
     * @param body  
     * @param subject  
     */  
    boolean sendEmail(string email, string body, string subject);  
  
    /**  
     *  
     * @param orderID  
     * @param status  
     */  
    boolean getOrderUpdate(in orderID, string status);  
  
}
```

```
public interface ISALoginAPI {  
  
    /**  
     *  
     * @param username  
     * @param password  
     */  
    boolean merchantLogin(string username, string password);  
  
    /**  
     *  
     * @param merchantID  
     */  
    boolean merchantDisconnect(int merchantID);  
  
}
```

```
public interface ISAMemberAPI {
```

Commercial-in-Confidence

```
/**
 *
 * @param regNumber
 * @param type
 * @param address
 * @param email
 * @param details
 */
boolean sendCommercialApplication(int regNumber, string type, string address, string
email, string details);
}
```

```
public interface ISAOrderAPI {
```

```
/**
 *
 * @param productID
 * @param quantity
 */
boolean placeOrder(int productID, int quantity);
```

```
/**
 *
 * @param orderID
 */
string trackOrderProgress(int orderID);
```

```
/**
 *
 * @param merchantID
 */
int queryBalance(int merchantID);
```

```
/**
 *
 * @param merchantID
 */
string[] viewPreviousOrders(int merchantID);

string[] getCatalogue();
```

```
/**
 *
 * @param merchantID
 */
string[] viewInvoices(int merchantID);
```

```
}
```

```
package IPOS_SA_Fragment.IPOS_SADetailedModel;
```

```
public class LoginAPIImpl implements ISALoginAPI {
```

```
    /**
     *
     * @param username
     * @param password
     */
    public boolean merchantLogin(string username, string password) {
        // TODO - implement LoginAPIImpl.merchantLogin
        throw new UnsupportedOperationException();
    }
```

```
    /**
     *
     * @param merchantID
     */
    public boolean merchantDisconnect(int merchantID) {
        // TODO - implement LoginAPIImpl.merchantDisconnect
        throw new UnsupportedOperationException();
    }
```

```
    public LoginAPIImpl() {
        // TODO - implement LoginAPIImpl.LoginAPIImpl
        throw new UnsupportedOperationException();
    }
```

```
}
```

```
package IPOS_SA_Fragment.IPOS_SADetailedModel;
```

```
public class MemberAPIImpl implements ISAMemberAPI {
```

```
    /**
     *
     * @param regNumber
     * @param type
     * @param address
     * @param email
     * @param details
     */
```

```
    public boolean sendCommercialApplication(int regNumber, string type, string address,
string email, string details) {
```

Commercial-in-Confidence

```
        // TODO - implement MemberAPIImpl.sendCommercialApplication
        throw new UnsupportedOperationException();
    }

    public MemberAPIImpl() {
        // TODO - implement MemberAPIImpl.MemberAPIImpl
        throw new UnsupportedOperationException();
    }
}
```

```
package IPOS_SA_Fragment.IPOS_SADetailedModel;

public class OrderAPIImpl implements ISAOOrderAPI {

    /**
     *
     * @param productID
     * @param quantity
     */
    public boolean placeOrder(int productID, int quantity) {
        // TODO - implement OrderAPIImpl.placeOrder
        throw new UnsupportedOperationException();
    }

    /**
     *
     * @param orderID
     */
    public String trackOrderProgress(int orderID) {
        // TODO - implement OrderAPIImpl.trackOrderProgress
        throw new UnsupportedOperationException();
    }

    /**
     *
     * @param merchantID
     */
    public int queryBalance(int merchantID) {
        // TODO - implement OrderAPIImpl.queryBalance
        throw new UnsupportedOperationException();
    }

    /**
     *
     * @param merchantID
     */
    public String[] viewPreviousOrders(int merchantID) {
```

```

        // TODO - implement OrderAPIImpl.viewPreviousOrders
        throw new UnsupportedOperationException();
    }

    public String[] getCatalogue() {
        // TODO - implement OrderAPIImpl.getCatalogue
        throw new UnsupportedOperationException();
    }

    /**
     *
     * @param merchantID
     */
    public String[] viewInvoices(int merchantID) {
        // TODO - implement OrderAPIImpl.viewInvoices
        throw new UnsupportedOperationException();
    }

    public OrderAPIImpl() {
        // TODO - implement OrderAPIImpl.OrderAPIImpl
        throw new UnsupportedOperationException();
    }
}

```

From these snippets of Python code, one can generate unit tests, if necessary for each of them, and develop **test cases** before the code of the implementation classes are available.

7.6 Testing the communications between subsystems

Now that we have defined the interfaces necessary for communication between the subsystems of IPOS, and generated Python code for all of them, we develop tests for inter-subsystem communication, i.e. of the respective implementation classes.

OrderAPIImpl

Method	Test ID	Parameter values	Comment	Expected outcome
placeOrder(productid:int, quantity:int):boolean	SA-O RD-01	productID= 100 quantity= 1	Enter valid int values for both parameters	Boolean operation so if successful, will return 'true' and order is created.
placeOrder(productid:int, quantity:int):boolean	SA-O RD-02	productID= 200 quantity= 1	Product may not exist in our inventory so invalid	Failure so will return 'false'

Commercial-in-Confidence

placeOrder(productid:int, quantity:int):boolean	SA-O RD-03	productID= -1 quantity= 1	Invalid product ID	Failure so will return 'false'
placeOrder(productid:int, quantity:int):boolean	SA-O RD-04	productID= 100 quantity= 10000000	Extremely large quantity - could be more than legally allowed to buy or more than our inventory	Depends on our stock levels. Would be 'true' if order can be fulfilled or 'false' if else
placeOrder(productid:int, quantity:int):boolean	SA-O RD-05	productID= 100 quantity= 0	Boundary value - quantity of zero	Failure so will return 'false'
placeOrder(productid:int, quantity:int):boolean	SA-O RD-06	productID= 100 quantity= -1	Negative quantity	Failure so will return 'false'

Method	Test ID	Parameter values	Comment	Expected outcome
trackOrderProgress(orderID:int):string	SA-O RD-07	orderID = 1000	Valid int value for orderID, AND exists in system as an already placed order	Success so will return 'true' representing a status like 'processing'
trackOrderProgress(orderID:int):string	SA-O RD-08	orderID = 999999	Order does not exist and has not been previously placed	Failure so will return 'false' representing 'order not found'
trackOrderProgress(orderID:int):string	SA-O RD-09	orderID = -1	Invalid order ID	Failure so will return 'false' representing 'invalid order ID'
trackOrderProgress(orderID:int):string	SA-O RD-10	orderID = 0	Boundary value not valid input	Failure so will return 'false'

Method	Test ID	Parameter values	Comment	Expected outcome
queryBalance(merchantID:int):int 11	SA-O RD-11	merchantID = 1000	Valid int value for merchant ID, AND merchant is found on the system	Success so will return valid balance as an int value
queryBalance(merchantID:int):int 11	SA-O RD-12	merchantID = 9999	Merchant does not exist	Failure so will return -1
queryBalance(merchantID:int):int 11	SA-O	merchantID = -1	Invalid order ID	Failure so will return -1

Commercial-in-Confidence

D:int):int 11	RD-13			
queryBalance(merchantID:int):int 11	SA-O RD-14	merchantID = 0	Boundary value not valid input	Failure so will return -1

Method	Test ID	Parameter values	Comment	Expected outcome
viewPreviousOrders(merchantID:int): string	SA-O RD-15	merchantID = 1000	Merchant has orders in system	Success so will return list of orders
viewPreviousOrders(merchantID:int): string	SA-O RD-16	merchantID = 1000	Merchant exists but does not have orders	Success so returns empty lists
viewPreviousOrders(merchantID:int): string	SA-O RD-17	merchantID = 2000	Merchant does not exist	Failure but returns empty list or error
viewPreviousOrders(merchantID:int): string	SA-O RD-18	merchantID = -1	Invalid ID	Failure

Method	Test ID	Parameter values	Comment	Expected outcome
getCatalogue():string	SA-O RD-19	none	Catalog populated	Success so will return list of products
getCatalogue():string	SA-O RD-20	none	Catalog empty	Success but returns an empty list

LoginAPIImpl

Method	Test ID	Parameter values	Comment	Expected outcome
merchantLogin(username,password):boolean	SA-L OG-01	User = "merchant1" Password = "correctPassword"	Valid credentials	Success so returns 'true'
merchantLogin(username,password):boolean	SA-L OG-02	User = "merchant1" Password = "wrongPassword"	Wrong password	Failure so will return 'false'

Commercial-in-Confidence

merchantLogin(username,password):boolean	SA-L OG-03	User = "unknown" Password = "pass"	Merchant does not exist	Failure so will return 'false'
merchantLogin(username,password):boolean	SA-L OG-04	User = "" Password = "pass"	Empty username	Failure so will return 'false'
merchantLogin(username,password):boolean	SA-L OG-05	User = "merchant1" Password = ""	Empty password	Failure so will return 'false'
merchantLogin(username,password):boolean	SA-L OG-06	User = "" Password = ""	Both empty	Failure so will return 'false'

Method	Test ID	Parameter values	Comment	Expected outcome
merchantDisconnect(merchantID:int):boolean	SA-L OG-07	merchantID = 1000	Valid merchant logged in	Success so returns 'true'
merchantDisconnect(merchantID:int):boolean	SA-L OG-08	merchantID = 1000	Merchant not logged in	Failure so will return 'false'
merchantDisconnect(merchantID:int):boolean	SA-L OG-09	merchantID = 9999	Merchant does not exist	Failure so will return 'false'
merchantDisconnect(merchantID:int):boolean	SA-L OG-010	merchantID = -1	Invalid ID	Failure so will return 'false'

MemberAPIImpl

Method	Test ID	Parameter values	Comment	Expected outcome
sendCommercialApplication()	SA-M EM-01	All valid parameters	Valid application	Success so returns 'true'
sendCommercialApplication()	SA-M EM-02	regNumber invalid	Invalid registration number	Failure so will return 'false'

Commercial-in-Confidence

sendCommercialApplication()	SA-MEM-03	Email invalid	Invalid email format	Failure so will return 'false'
sendCommercialApplication()	SA-MEM-04	Type empty	Missing type	Failure so will return 'false'
sendCommercialApplication()	SA-MEM-05	Address empty	Missing address	Failure so will return 'false'
sendCommercialApplication()	SA-MEM-06	Details empty	Missing details	Failure so will return 'false'

Method	Test ID	Parameter values	Comment	Expected outcome
checkCommercialApplication(regNumber:int):boolean	SA-MEM-07	regNumber = 1000	Application exists	Success so returns 'true'
checkCommercialApplication(regNumber:int):boolean	SA-MEM-08	regNumber = 9999	Application does not exist	Failure so will return 'false'
checkCommercialApplication(regNumber:int):boolean	SA-MEM-09	regNumber = -1	Invalid regNumber	Failure so will return 'false'

IPUAPIImpl

Method	Test ID	Parameter values	Comment	Expected outcome
sendEmail(email,body,subject):boolean	SA-IPU-01	All valid parameters	Valid email	Success so returns 'true'
sendEmail(email,body,subject):boolean	SA-IPU-02	Invalid email format	Invalid email	Failure so will return 'false'
sendEmail(email,body,subject):boolean	SA-IPU-03	Empty email	Missing recipient	Failure so will return 'false'

Commercial-in-Confidence

sendEmail(email,body,subject):boolean	SA-IP U-04	Empty subject	Missing subject	Failure so will return 'false'
sendEmail(email,body,subject):boolean	SA-IP U-05	Empty body	Missing body	Failure so will return 'false'
sendEmail(email,body,subject):boolean	SA-IP U-06	All empty	Completely invalid	Failure so will return 'false'

8 Appendices

8.1 Initial Statement of requirements by the client

IPOS: Info Pharma Ordering System

The InfoPharma Ordering System (IPOS) will keep track of the orders of pharmaceutical goods made by merchants to InfoPharma Ltd and will assist the merchants' sales to consumers.

IPOS includes **three** subsystems:

- I. A server application (IPOS-SA), which in the fully functional production version will be administered by the IT department of InfoPharma Ltd.
- II. A client application (IPOS-CA), a copy of which will be run by the merchant on a desktop PC in the merchant's Pharmacy shop.
- III. An online platform (IPOS - PU) to be used by a merchant for direct sales of pharmaceutical goods to the public. The platform will allow the merchants to run advertisement/promotion campaigns with new products with discounts.

In the prototype of IPOS the three subsystems should be implemented as separate **desktop applications** able to interact with one another as necessary to meet the requirements. For instance, online sales (via IPOS - PU) should be linked to the inventory of the merchant: successful online purchases should be deducted from the stock of goods available at merchant's shop. Likewise, orders from IPOS-SA and delivered to a merchant, should be easy to record in IPOS-CA and increase the merchant's stock. Finally, promotion campaigns run via IPOS – PU may affect the cost of selling goods online (at reduced price), but not the cost of the same goods if sold in the pharmacy shops.

8.2 Hardware (minimal and optimal configuration)

8.2.1 minimal hardware configuration

The minimal hardware configuration recommended for IPOS development consists of a desktop PC, 1GB of RAM and at least 50GB of disk space for storing the software programs and data possibly running a database server such as MySQL or other similar. A hosting package with preinstalled software and at least 1 GB for use data is also acceptable for both development and live deployment of IPOS.

8.2.2 optimal hardware configuration

The optimal hardware configuration envisaged for development should consist of a desktop PC, 4GB of RAM and at least 250GB of disk space for storing the software programs and data possibly running a database server such as PostgreSQL or better. Again, a hosting package

Commercial-in-Confidence

with preinstalled software + 5GB of data is considered an optimal hardware configuration for both development and live deployment

9 Index

Not provided in this release of the document